

O que você escreve não é o que sobe

CÓDIGO FONTE

```
function calcularTotal(itens) {  
  // soma o preço de cada item  
  let total = 0;  
  for (const item of itens) {  
    total += item.preco;  
  }  
  return total;  
}
```

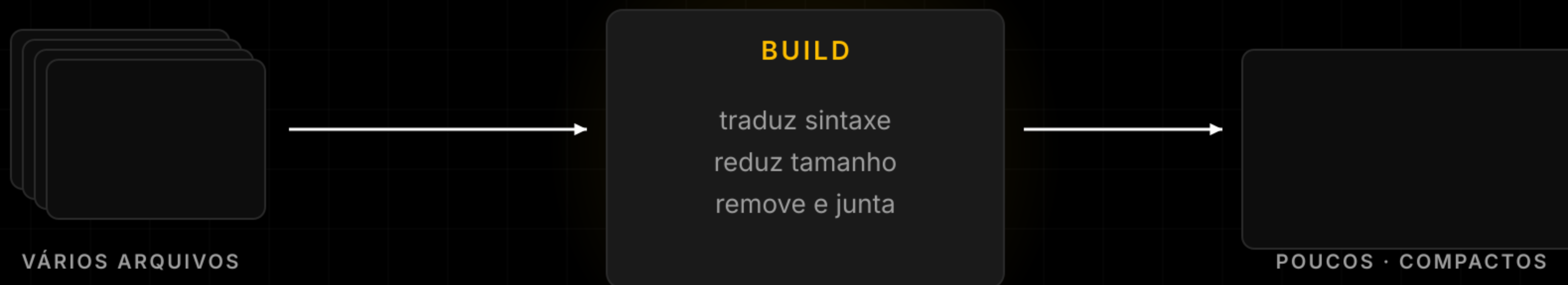
DEPOIS DO BUILD

```
function a(b){let c=0;  
  for(const d of b)c+=d.preco;  
  return c}
```

sem comentários · nomes curtos
compacto para a máquina

Você escreve para humanos. O build reescreve para a máquina.

O que o build realmente faz



Nem todo build faz tudo. Mas esses três nomes vão aparecer.

Três motivos de build quebrado

SINTAXE INVÁLIDA



a gramática do código

parêntese aberto, vírgula faltando

ERRO DE TIPO



categoria do valor

número tratado como texto

DEPENDÊNCIA FALTANDO

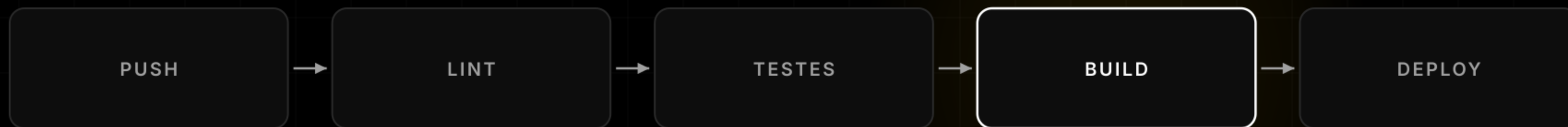


código de outra pessoa

usou e não declarou, ou versão errada

Build failed: a transformação não terminou. Leia o arquivo e a linha.

O cano, etapa por etapa



A PRIMEIRA PORTA

Cada etapa é uma porta. Vermelho em qualquer uma: nada segue adiante.

O build é a primeira porta. Se ele não passa, nada passa.

CI: o portão que abre sozinho



Humano não confere na mão. O cano confere sozinho.

Três verificações, três proteções

LINT

estilo

como o time combinou escrever

indentação · nomes · aspas

TESTES

comportamento

se X, espero Y

o que você prometeu continua

BUILD

transformação

consegue virar versão servível

se quebra aqui, sinal forte

CI vermelho: alguém lê o erro, conserta, e o cano tenta de novo.

CD: duas fronteiras de automação

CONTINUOUS DELIVERY

prepara e para

versão publicável pronta
humano aperta o botão

PORTÃO HUMANO

CONTINUOUS DEPLOYMENT

publica sozinho

CI verde → já vai ao ar
sem decisão humana

AUTOMÁTICO

Os dois se chamam CD. O que muda é quem decide publicar.

Quatro ambientes, um caminho



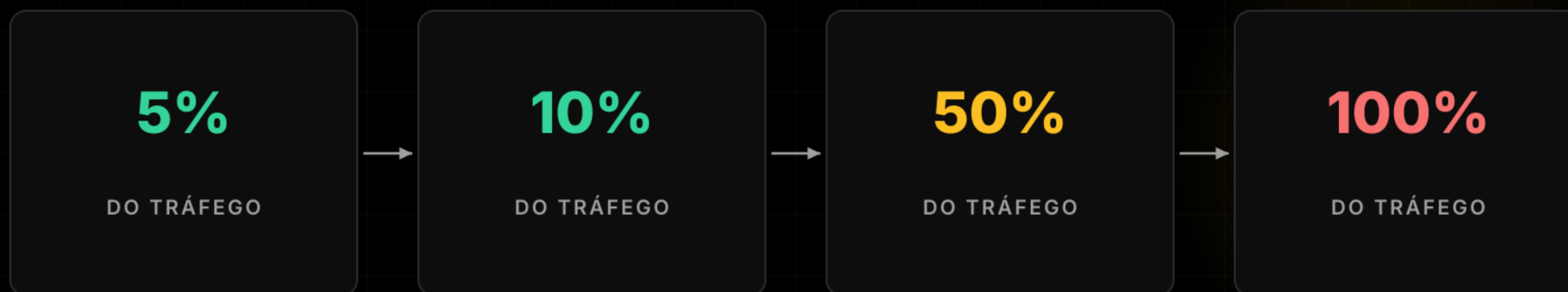
O código sobe de ambiente em ambiente. Prod pede mais cuidado.

Blue-green: sobe ao lado e troca



A antiga fica lá. Se der problema, o tráfego volta sem reescrever.

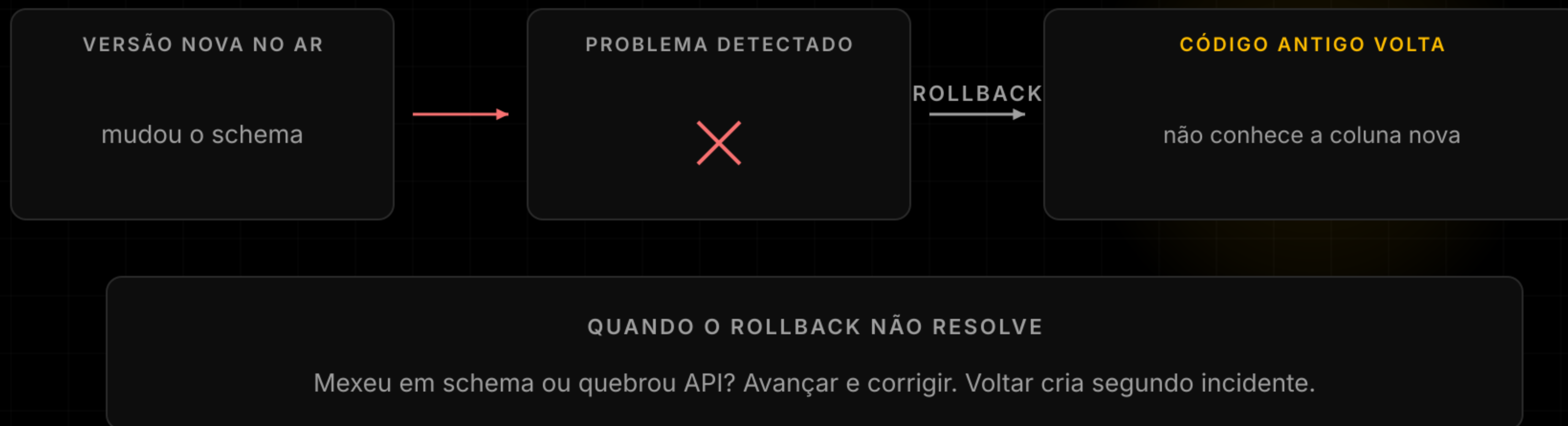
Canary: libera para poucos primeiro



Se quebrar no começo, só poucos perceberam. E você volta rápido.

Estratégia existe para reduzir o risco de publicar.

Rollback + schema: voltar pode piorar



Rollback troca versão. Não desfaz mudança de dado.

Incidente e post-mortem

INCIDENTE

quebra com impacto real

não consegue logar
não consegue comprar
não consegue acessar

POST-MORTEM

- o que aconteceu?
- como detectamos?
- como resolvemos?
- o que vamos mudar?

Controlou o problema. Depois senta e pergunta para não repetir.

O ciclo completo, até o usuário



Lint, testes e build moram dentro do CI. Blue-green e canary, no deploy.

O usuário só enxerga a última ponta. E percebe se algo mudou.

Se deu ruim: rollback (quando seguro), entender, corrigir, de novo.

O mapa da temporada, fechado

EP 02

front-end e estado

navegador · HTML · CSS · JS

EP 03

request, response, API

a ponte navegador ↔ servidor

EP 04

banco de dados

memória de longo prazo

EP 05

auth e sessão

quem é você · o que pode

EP 06

git e versionamento

fotos no tempo · rede de segurança

EP 07

build, CI/CD, deploy

o cano que leva ao ar

Quando a IA sugerir mudança: em qual camada isso mora?